SuperMimic on Kangaroo, M2: the world model

Offline study of a learned dynamics model for PAL Kangaroo (7-DoF arms). Branch `omm/supermimic` of pal_mjlab, 23 October 2026.

Summary

SuperMimic (arXiv 2610.28480) trains a motion-tracking policy by backpropagating through a learned model of the robot's dynamics instead of through the simulator. Milestone M2 tested that model alone, before any policy is trained on it. We fit the paper's world model, an MLP with three layers of 2048 units, on 6.1M simulated Kangaroo transitions logged from our PPO baseline. Then we measured it on motion clips it never saw during training.

- **It predicts well up to the paper's horizon of 32 steps (0.64 s).** Run open loop on recorded actions, the 14 tracked bodies are off by **0.86 cm after 8 steps** and **3.8 cm after 32 steps**. That is 4–6× below a baseline that predicts no motion. The imagined tracking return over 32 steps is within **1.1%** of the real one. At 64 steps the error grows to 15 cm, so 32 steps is about the useful limit.
- **Its gradients improve the real return.** A small change of the actions along the model's gradient raised the simulated return from **97–100%** of the start states. Random changes of the same size did so from 44–47%. Up to 16 steps the predicted gain matches the real one within 10%. At 32 steps the model is **optimistic**: it promises 1.3–1.6× the gain that is realized.
- **The paper's choices hold on Kangaroo.** Training on 8-step windows is essential: a model trained on single steps *diverges* at 32 steps. Predicting state changes is 2× better than predicting the next state. The full width of 2048 units is 15–20% better than 1024. A policy the model never saw costs about 15%.
- **The learned failure detector is the weak part.** It ranks failures with AUC 0.937, below our 0.95 gate. The simulator's own failure rule, applied to the model's predicted states, reaches 0.995.

Recommendation for M3. Use the main configuration (8-step training, 2048×3, delta targets). In imagination, replace the learned failure head with a smooth version of the simulator's failure rule. Log the imagined-vs-simulated return gap from the first iteration, because the model is optimistic at 32 steps. Total GPU time for M2 was about 2.6 h on the local RTX 5060 Ti.

1 Why the world model is the main risk

The SuperMimic loop. The paper repeats three steps (Alg. A.1):

1. **Collect** simulator transitions with the current policy plus Gaussian action noise $\mathcal{N}(0, 0.1^2)$, into a replay buffer.
2. **Fit the world model** f_ω on short windows from the buffer (paper Eq. 1).
3. **Train the policy** by imagining $H = 32$ steps with the model, from start states drawn from the buffer, and backpropagating the imagined return into the policy (paper Eq. 2). A critic V estimates the return beyond the horizon (Eq. 3).

The imagined return. With predicted states $\hat{s}$, reference goals g, rewards r and a survival weight c that discounts states after a likely fall, the policy maximizes

$$J(\theta) = \sum_{t=0}^{H-1} \gamma^t \, c_t \, r(\hat{s}_{t+1}, g_{t+1}) + \gamma^H c_H \, V(\hat{s}_H), \qquad \hat{s}_{t+1} = f_\omega\big(\hat{s}_t,\ \pi_\theta(o_t, g_t),\ \xi\big), \qquad c_{t+1} = c_t \, (1 - \hat{d}_{t+1}),$$

where ξ is the episode's domain-randomization context (friction, calibration, centre of mass) and $\hat{d}$ the predicted failure probability.

The model is not a cheaper simulator. Its job is to be *differentiable*. The two methods get the update direction $\nabla_\theta J$ in different ways:

- **PPO tries and sees.** A stochastic policy samples actions, the simulator returns rewards, and the gradient is estimated as $\mathbb{E}[\nabla_\theta \log \pi_\theta(a \,|\, o)\, \hat{A}]$. It never differentiates the dynamics. Each rollout only says whether the sampled actions were better or worse than average, so many rollouts are averaged to beat the noise.

- **SuperMimic differentiates.** The policy is deterministic, and J is an explicit chain of differentiable functions: policy, model, forward kinematics, reward, and the critic at the end. Autograd returns the exact $\nabla_\theta J$ in one backward pass over the 32 steps. Through the chain rule this contains the model's Jacobians, for example

$$\frac{\partial r_t}{\partial a_0} = \frac{\partial r_t}{\partial \hat{s}_t} \frac{\partial \hat{s}_t}{\partial \hat{s}_{t-1}} \cdots \frac{\partial \hat{s}_2}{\partial \hat{s}_1} \frac{\partial \hat{s}_1}{\partial a_0}.$$

- **Why not the simulator.** MuJoCo gives no such derivatives here (we run it without a backward pass), and derivatives through contacts are poorly behaved even in differentiable simulators. The learned model is a smooth network, so its Jacobians exist and are smooth across contacts.

Each imagined rollout therefore carries an exact gradient for every action, not one noisy return. That is where the paper's efficiency comes from.

Why this is the risk. That gradient is only as good as the model's *slopes*. The model's predictions must be right, *and* its derivatives with respect to the actions must point the way the real dynamics would. If either fails, the policy improves in imagination and not in reality. Kangaroo makes this harder than the paper's G1:

- its legs are closed kinematic chains (a passive four-bar knee);
- its arms are heavier and use soft 1 Hz gains (see the M1 gain study);
- its hips and ankles have narrower limits.

The M0 plan therefore ranked the world model as risk number 1 and gave it its own milestone. M2 does not train a policy. It fixes a data set, trains models on it, and asks:

Q1 How far ahead does the model stay accurate, in the quantities the reward uses?

Q2 Do its action gradients improve the *real* return?

Q3 Can it predict failures, which set the survival weight c?

Q4 Which of the paper's design choices matter on Kangaroo, and which configuration goes to M3?

2 The model

2.1 State: what the model predicts

The simulator state is the floating-base pose and velocity plus 32 joint positions and velocities ($q \in \mathbb{R}^{39}$, $\dot{q} \in \mathbb{R}^{38}$). On flat ground the dynamics do not depend on where the robot stands or which way it faces. So the model sees a *heading-invariant* state with the world position and yaw removed (Table 1).

Table 1: The 77-dimensional model state s. The heading frame is the world frame rotated by the root's yaw.

component	dims	frame / note
root height z	1	world
root tilt (rotation without yaw, 6D: two matrix columns)	6	heading frame
root linear velocity	3	root frame
root angular velocity	3	root frame
joint positions, including the 4 passive four-bar joints	32	
joint velocities	32	
total	77	

- **World pose outside the network.** The reward compares body positions with the reference in the world, so the world pose $p = (x, y, \psi)$ must still be tracked. The network predicts the root's displacement and yaw change in its own heading frame. The pose is then integrated analytically: $p_{t+1} = p_t \oplus \Delta p_t$.
- **Exact round trip.** The map $(s, p) \leftrightarrow (q, \dot{q})$ is exact and differentiable; tests check it to 10^{-10}.
- **Passive joints are in the state.** The four-bar's femur and knee angles are part of the mechanism's state; without them the state would not be Markov. They never reach the policy's inputs (D28).

2.2 Dynamics network

- **Inputs:** the state s_t, the action a_t and the context ξ.
  - a_t: 28 actuated joints, in action units; the setpoint is $q_0 + \alpha a$.
  - ξ: 32 dims, namely foot friction (1), encoder calibration offsets (28) and the torso centre-of-mass offset (3).

  Every input dimension is standardized with a mean and standard deviation computed once from 200k training transitions and then frozen.
- **Network:** an MLP with three hidden layers of 2048 ELU units (paper Table A.4).
- **Outputs (80):** the standardized *change* of the state (77), and the root's heading-frame displacement and yaw change (3). The next state is $\hat{s}_{t+1} = s_t + \Delta\hat{s}_t$.
- **Size:** 8.9M parameters, including the termination head below.

2.3 Reward and observations are computed, not learned

- **The reward is a function of the state.** The env's tracking reward (paper Table A.2) scores the position, orientation, linear and angular velocity errors of 14 tracked bodies against the reference, through exponential kernels. Those body quantities follow from $(q, \dot{q})$ by forward kinematics.
- **So we compute it.** Our differentiable torch forward kinematics (built in M0, compiled) reproduces MuJoCo, so in imagination the reward is computed from the predicted state with the env's own reward function (D5).
- **Parity check.** The analytic reward on the logged states matches the reward the env logged, with a maximum difference of $1.6 \cdot 10^{-5}$.
- **Consequence.** Any reward error in imagination is purely a state-prediction error. M3 computes the policy's observations the same way.

2.4 Termination head

- **What fails an episode.** The env's rule (paper): the root height is more than 0.25 m from the reference's, or the vertical component of the projected gravity, g_z, is more than 0.8 from the reference's.
- **The head.** A small MLP (256×2) predicts this failure as a probability $\hat{d}$. Its inputs are the predicted state and the reference root height and g_z, which is everything the rule needs.
- **Training:** binary cross-entropy on the logged failure flags.

2.5 Training loss (paper Eq. 1)

A training example is a window of k consecutive logged transitions. The model starts from the first logged state and runs k steps *on its own predictions*, driven by the logged actions. The loss compares every predicted step with the log:

$$\mathcal{L}(\omega) = \frac{1}{k}\sum_{j=1}^{k}\Big[\underbrace{\tfrac{1}{77}\big\|(\hat{s}_j - s_j)/\sigma_s\big\|^2}_{\text{state}} + \underbrace{\frac{\|e_{xy,j}\|^2}{(0.1\,\text{m})^2} + \frac{e_{\psi,j}^2}{(0.1\,\text{rad})^2}}_{\text{world pose}} + \underbrace{\text{BCE}(\hat{d}_j, d_j)}_{\text{failure}}\Big].$$

- σ_s is the per-dimension state standard deviation.
- e_{xy} and e_ψ are the errors of the integrated planar position and yaw, measured in the true heading frame.
- Gradients flow through the whole k-step rollout. The model is therefore trained to correct the drift of its own predictions, which single-step training ($k = 1$) never sees.
- Optimization (paper settings): $k = 8$; Adam with learning rate $2 \cdot 10^{-4}$; batches of 2048 windows; global gradient-norm clipping at 1.0; 50k steps. We keep the checkpoint with the best held-out 8-step loss.

3 Data

- **Source policies.** Three checkpoints of the 12 h PPO baseline from M1, at iterations 1000, 4000 and 8800 (the final one). Their skill levels differ, so the data covers good tracking, poor tracking and falls.

- **Collection.** 4096 parallel envs per checkpoint, run for 500 control steps (10 s) on the `dev` library: 1296 BONES-SEED clips, 2.56 h of motion. Settings, as in the paper:
  - the deterministic policy plus action noise $\mathcal{N}(0, 0.1^2)$;
  - the domain randomization drawn per episode;
  - return-based reset sampling.

  The whole collection took 2 minutes.
- **Terminal states are kept.** Automatic resets are off, so the true last state of each episode is logged before the env resets it. Training windows never span an episode boundary, so the model never sees a teleport (unit-tested).
- **Held-out clips.** 10% of the clips (118 of 1296), chosen by a hash of the clip name, are removed from training in every data set. All results below are on these clips. Training uses the other 5.28M valid 8-step windows.

Table 2: Collected data. Failures are 0.05% of transitions, which matters for the termination head.

policy (PPO iteration)	transitions	episodes ended	failures
1000	2.05M	9662	1480
4000	2.05M	8006	417
8800 (final)	2.05M	7945	1073
total	6.14M	25613	2970

4 How we measured

4.1 Open-loop prediction

Protocol. For each horizon $h \in \{1, 8, 16, 32, 64\}$ steps (20 ms each), we draw 4096 random held-out windows. Each model starts from the logged state and pose and applies the logged actions h times with no correction. At step h we compare with the logged state:

- **Body error:** the world-position error of the 14 tracked bodies after forward kinematics, averaged over bodies (cm). This is what the reward sees. We also report its 95th percentile over windows.
- **Local body error by group** (arms, legs, torso): body positions relative to the root. This separates posture errors from root drift.
- **Root errors:** height, tilt angle and planar position.
- **Return error:** $|\sum \hat{r} - \sum r| / \sum r$ over the window, with both rewards computed by the env's reward function.

Floor. A "no change" predictor keeps the start state and pose. Its error shows how much the robot actually moves over h steps. A useful model has to beat it by a wide margin.

4.2 Gradient check

Question. Does a step along the model's gradient $\partial J/\partial a$ improve the *simulated* return?

Protocol. Take 256 held-out start states with their logged action sequences a ($H \times 28$).

1. **Imagined return.** $J(a)$ is Eq. 2 without the critic: $\gamma = 0.99$, and the survival weight comes from the termination head.
2. **Gradient.** Backpropagate through the model to get $\partial J/\partial a$.
3. **Gradient steps.** $a' = a + \delta\sqrt{28H}\ \partial J/\partial a \,/\, \|\partial J/\partial a\|$, so every action entry changes by δ in RMS. We use $\delta = 0.05$ and 0.1 action units; for scale, the collection noise is 0.1.
4. **Controls.** Two random directions of the same size per δ.
5. **Replay.** Every sequence is replayed open loop in the simulator from the same start state, with the same context ξ (friction, calibration and centre of mass written into the simulator) and the same reference frame. The simulated return uses the env's tracking reward.

Replay fidelity. Replaying the logged actions reproduces the logged return to within 0.003 (of about 98 at $H = 32$), so the simulator side is exact.

Reported: the share of starts that improved, the mean predicted and real gains, their correlation across starts, and the gradient norm per action step along the horizon.

4.3 Failure prediction

- **Test set.** Every held-out 8-step window ending in a failure, plus four times as many random windows. Every step of every window is scored: 8800 steps, 220 of them failures.
- **The two scores,** both on the model's *predicted* states:
  - the learned head's logit;
  - the margin of the env's rule, $m = \max\left(|z - z_{\text{ref}}|/0.25\,\text{m},\ |g_z - g_{z,\text{ref}}|/0.8\right)$, where $m > 1$ means failure.
- **Metric:** the AUC, i.e. the probability that a failure step scores higher than a non-failure step.

5 Results

5.1 Q1: accuracy of the main model



Figure 1: Open-loop error on held-out clips vs prediction horizon (log scales). Left: mean world-position error of the 14 tracked bodies. Right: relative error of the imagined tracking return. "No change" is the floor; the k=1 model leaves the plot at 32 steps.

Table 3: Main model vs the no-change floor, held-out clips, 4096 windows per horizon. Errors at step h; return error over the window.

predictor	h steps	body cm	body p95 cm	arms (local) cm	legs (local) cm	root z cm	tilt deg	root xy cm	return err %
no change	1	0.73	2.10	0.74	0.49	0.18	0.46	0.50	1.10
	8	5.55	16.16	5.52	3.32	1.21	3.08	3.89	4.53
	16	10.20	31.03	9.53	5.12	1.75	4.65	7.49	8.36
	32	16.94	52.46	14.02	6.53	2.17	6.33	12.71	13.26
	64	26.43	88.42	17.45	6.88	2.28	7.09	20.92	17.49
main (k=8, 2048×3, Δ)	1	0.15	0.32	0.16	0.12	0.06	0.08	0.03	0.34
	8	0.86	1.82	1.20	0.56	0.16	0.49	0.25	0.54
	16	1.70	3.57	2.34	0.93	0.25	0.85	0.73	0.75
	32	3.83	8.19	4.37	2.11	0.35	2.06	2.64	1.08
	64	14.93	33.05	10.93	9.03	1.37	9.45	14.74	3.26

- **Error grows smoothly and stays small up to 32 steps:** 0.15, 0.86, 1.7 and 3.8 cm at 1, 8, 16 and 32 steps. That is 4.4–6.5× below the no-change floor. At 32 steps, 95% of windows are within 8.2 cm.
- **The root is accurate:** 0.35 cm in height, 2.1° in tilt and 2.6 cm of planar drift after 32 steps.
- **The reward is even more accurate:** the imagined return is within 1.1% at 32 steps. The exponential kernels barely react to errors of a few centimetres. What the policy needs is the reward's *gradient*, which Q2 tests.
- **Beyond 32 steps the error accelerates:** 14.9 cm at 64 steps (95th percentile 33 cm), only 1.8× better than no change. Every rollout stayed finite, but 64 steps is past the model's useful range. This agrees with the paper's $H = 32$ plus a critic for the rest.

Where the error is (Figure 2).

- **By group:** relative to the root, the arms carry the largest error (4.4 cm at 32 steps), the legs half of it (2.1 cm), and the torso, close to the root, very little (0.24 cm). In joint space, arm errors are 0.038 rad and leg errors 0.016 rad.
- **Likely reasons (untested):**
  - The arms move the most in BONES-SEED motions.
  - At 1 Hz they are compliant: where they go depends on how the torso swings them, not only on their setpoint.
- **Same pattern in PPO-Sim:** the arms were also the largest tracking error there (10.8 cm).

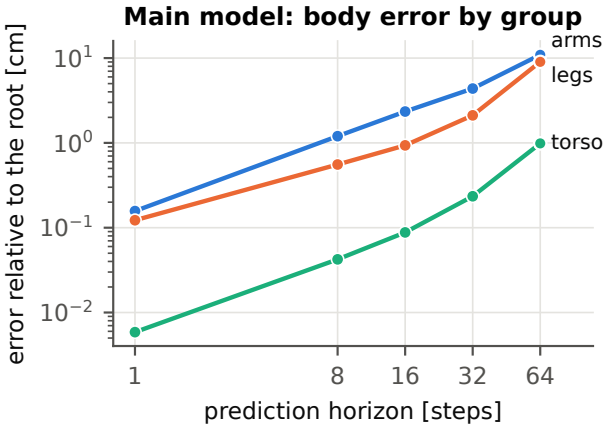


Figure 2: Main model: body-position error relative to the root, by group, vs horizon.

5.2 Q4: which design choices matter (ablations)

Each ablation changes one thing from the main model; data, split, steps and seed are the same. Table 4 and Figure 1 compare the results on held-out clips, and Figure 3 shows the held-out losses during training.

Table 4: Ablations on held-out clips: mean body error and imagined-return error. "Diverged": the mean error exceeds 10 m because a fraction of the rollouts blow up.

	8 steps		32 steps		64 steps
model	body [cm]	return [%]	body [cm]	return [%]	body [cm]
main (k=8, 2048×3, Δ)	0.86	0.54	3.83	1.08	14.9
width 1024	0.98	0.62	4.58	1.29	18.6
absolute targets	2.37	1.25	8.36	3.38	26.4
k=1 training	1.45	0.79	diverged	2.95	diverged
no change	5.55	4.53	16.94	13.26	26.4

- **Single-step training (k=1) diverges.**
  - After one step it is almost as good as the main model (0.19 vs 0.15 cm). Its error then compounds: 1.45 cm at 8 steps, and it diverges at 32 steps.
  - In training, its 32-step held-out loss explodes from about 30k steps on (Figure 3, right), while its 8-step loss keeps falling.
  - Why: trained only on true inputs, the model never learns what to do once its own small errors take it off the data, so nothing holds it there.
  - Its return error (3%) hides the divergence, because the reward saturates at zero for far-off states.

  The paper's multi-step loss is essential.
- **Absolute targets are 2× worse.** Predicting s_{t+1} directly instead of $s_{t+1} - s_t$ gives 0.46 cm after one step (3× worse) and 8.4 cm at 32 steps (2.2×). The network must reproduce the whole state to recover a small change, and the change drowns in the regression error.
- **Width 1024 is 14–19% worse.**
  - With 2.4M instead of 8.9M parameters: 0.98 cm at 8 steps and 4.6 cm at 32.
  - It trains 2.7× faster (16 vs 43 min).
  - The main model is still improving at 50k steps (Figure 3), so more steps or data may still help.
- **An unseen policy costs about 15% (Table 5).** We trained a model without the iteration-4000 data and tested it on that policy's held-out clips. Error rises from 0.82 to 0.93 cm at 8 steps and from 3.6 to 4.2 cm at 32. The model carries over even to a policy thousands of PPO iterations away. In the online loop the policy moves one small step between model refits.

Table 5: Policy shift: held-out clips of the iteration-4000 data, for the main model (trained on it) and a model trained only on the iteration-1000 and 8800 data.

model (data of the 4000-iteration policy)	body, 8 steps [cm]	body, 32 steps [cm]	return err, 32 [%]
main (k=8, 2048×3, Δ)	0.82	3.60	1.02
unseen policy	0.93	4.18	1.30

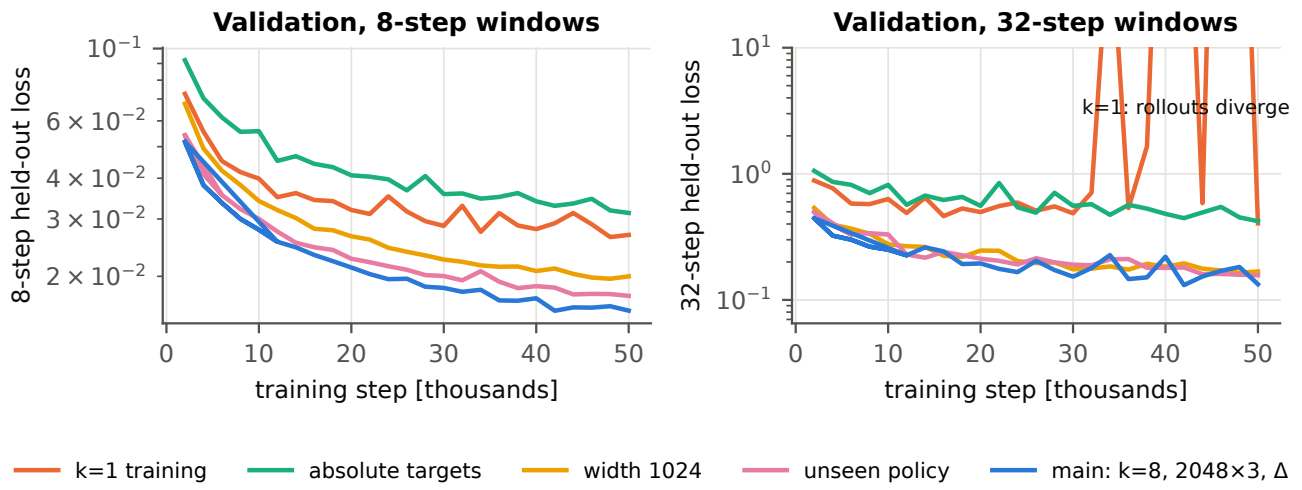


Figure 3: Held-out losses during training (Eq. 1 without the failure term). Left: 8-step windows, the training length. Right: 32-step windows, the policy's horizon. The k=1 model improves at 8 steps while its 32-step rollouts diverge.

Not ablated.

- **The "10 batches of 2048 windows" reading (D10).** It sets how many model updates the online loop makes per environment step. Offline it has no meaning, so it moves to M4.
- **The normalizer ablation.** Dropped: frozen per-dimension statistics trained every variant smoothly, and nothing pointed at them.

5.3 Q2: are the gradients useful?

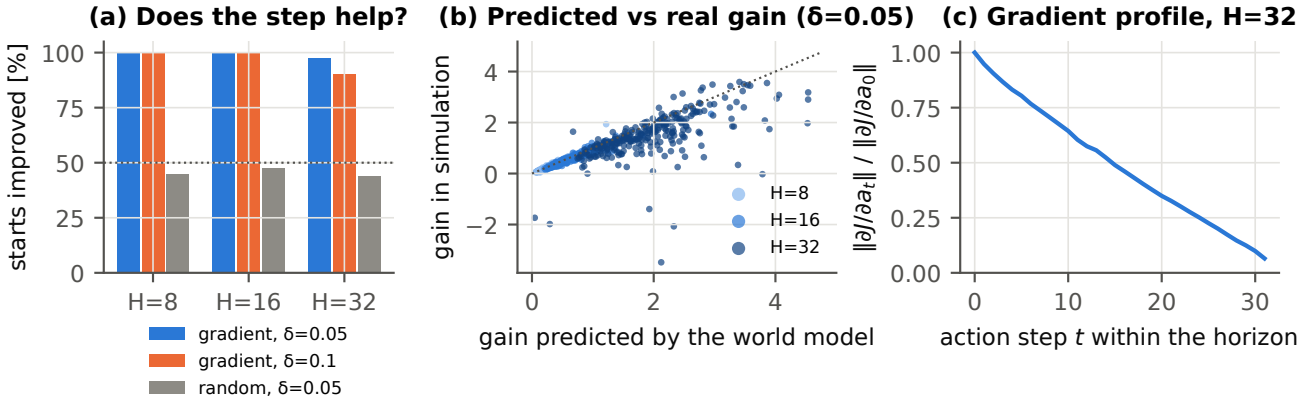


Figure 4: Gradient check, main model, 256 held-out starts. (a) Share of starts whose simulated return improved after a step along the model's gradient vs a random direction of the same size. (b) Gain predicted by the model vs gain measured in simulation, per start ($\delta = 0.05$); the dotted line is $y = x$. (c) Gradient norm per action step along a 32-step horizon, relative to the first step.

Table 6: Gradient check: share of improved starts, mean return gain predicted by the model and measured in simulation, and their correlation across starts. Random: mean of two directions, $\delta = 0.05$.

	starts improved [%]			$\delta = 0.05$			$\delta = 0.1$	
H	grad 0.05	grad 0.1	random	predicted	real	corr	predicted	real
8	100	100	45	0.24	0.23	0.95	0.43	0.41
16	100	100	47	0.68	0.63	0.96	1.18	1.07
32	97	90	44	1.98	1.50	0.62	2.70	1.72

- **Direction: the gradient is right almost every time.**
  - At $H = 8$ and 16, every gradient step improved the simulated return.
  - At $H = 32$: 97% of starts improved for $\delta = 0.05$, and 90% for $\delta = 0.1$.
  - Random directions improved 44–47%. The logged actions come from a trained policy, so a random change hurts slightly more often than it helps.
- **Size: accurate up to 16 steps, optimistic at 32.**
  - Up to 16 steps the predicted gain is within 10% of the real one, and the two correlate at 0.95–0.97 across starts.
  - At 32 steps the model predicts 1.98 where the simulator gives 1.50 ($\delta = 0.05$, +32%), and 2.70 vs 1.72 ($\delta = 0.1$, +57%). The correlation drops to 0.62–0.72.
  - The larger the step, the larger the optimism. This is the classic failure mode of model-based RL: an optimizer finds the actions where the model errs in its own favour. SuperMimic's policy update is exactly such an optimizer; the paper's online refits correct the model wherever the policy goes.
- **Gradient flow: no explosion, no vanishing** (Figure 4c).
  - The norm of $\partial J/\partial a_t$ decays smoothly and almost linearly along the horizon, from 0.54 at $t = 0$ to 0.035 at $t = 31$.
  - An action at step t can only affect the $H - t$ rewards after it, so a linear decay is what well-behaved dynamics give.
  - Exploding gradients would grow faster than linearly toward $t = 0$; vanishing gradients would collapse within a few steps. Neither happens.

The gains are small next to the return (1.5 of about 98 at $H = 32$) because the steps are deliberately small, i.e. first order. The test is about direction, and the direction is right.

5.4 Q3: predicting failures

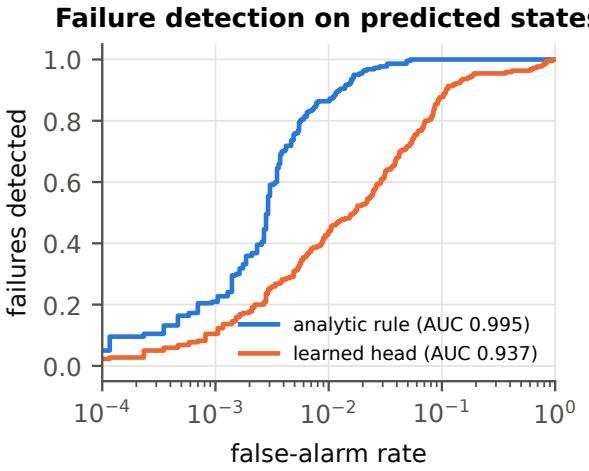


Figure 5: Failure detection on the main model's predicted states, held-out clips (log false-alarm axis). The analytic rule is the env's own failure definition applied to the predicted state.

- **The learned head misses the gate:** AUC 0.937, against a gate of 0.95. The other models score 0.91–0.95, except k=1 at 0.71.
- **The env's rule on the same predicted states scores 0.995** (0.97–0.996 across models).
- **Under policy shift** the head drops to 0.872 for the model that never saw the policy, while the rule stays at 0.997.
- **The fault is the head, not the states.** A larger check with 20× negatives gives the head 0.961 on predicted states and 0.954 on *true* states. Given perfect states, it is still worse than the rule.
- **The rule as a hard threshold** ($m > 1$) catches 81% of failures at the exact step, at a false-alarm rate of 0.02%.

Why the head struggles. Failures are 0.05% of transitions. The label is a sharp threshold on two quantities. Cross-entropy is minimized by answering a confident "no failure" almost everywhere: the held-out failure loss sits near 10^{-4} while the ranking stays mediocre. The rule, in contrast, *is* the definition of the label, and the predicted states are accurate enough to apply it.

5.5 Gates

Table 7: M2 gates (main model, held-out clips).

question	gate	measured	
Q1	8-step body error < 2 cm mean, < 5 cm p95	0.86 / 1.82 cm	pass
Q1	32-step rollouts bounded, body error < 10 cm	3.83 cm, all finite	pass
Q1	imagined return error < 10% at 32 steps	1.1%	pass
Q2	gradient steps beat random directions	90–100% vs 44–47%	pass
Q3	failure AUC > 0.95	head 0.937 / rule 0.995	head fails, rule passes
Q4	k=8 beats k=1	3.8 cm vs diverged at 32 steps	pass

6 Decisions for M3 and M4

1. **World model:** the main configuration: $k = 8$, 2048×3 ELU, delta targets, frozen normalization. M3 uses the checkpoint trained on all three data sets (`logs/supermimic_m2/wm/main/wm_best.pt`); M4 trains the same configuration online from scratch.

2. **Failure probability in imagination: a smooth analytic rule**, in place of the learned head: $\hat{d} = \sigma\big((m(\hat{s}, g) - 1)/\tau\big)$ with $\tau \approx 0.05$.
   - It is differentiable, so the survival weight still pushes the policy away from falls.
   - It has no parameters to fit, and it does not degrade under policy shift.
   - It deviates from the paper, which learns d_ω. The learned head keeps training next to it and its AUC is logged online, so no separate run is needed to compare them.
3. **Guard against model optimism** at $H = 32$:
   - Log the imagined vs simulated return of the same policy from the first iteration.
   - Keep the paper's small policy steps: learning rate $2 \cdot 10^{-4}$, gradient clipping at a global norm of 1.0 and then 0.1 per element.
   - If the gap grows in M4, ablate $H = 16$, where the gradients were accurate.
4. **Watch the arms.** They carry the largest model error, and they were already the weakest part of the PPO baseline.

7 Next steps

No policy is trained to convergence on a frozen model. The paper never does it. In Alg. A.1 the model is refit every iteration on data from the current policy. It is frozen only within a policy step, in the sense that the policy loss sends no gradient into ω. Those refits are what stop the policy from exploiting model errors. A policy trained for long on the M2 model would leave its data and exploit it, and that run would fail for a reason the paper never meets. So M3 is a short check of the new code, and every comparison between methods moves to the online loop (M4).

M3: check the imagination code (about an hour of GPU). Build:

- **Imagination:** roll out from replay start states, with the previous 9 logged steps to fill the 10-frame observation history.
  - Observations are computed from the predicted state (actuated joints only, D28), with the sensor noise added back as in the paper.
  - Goals are the next 10 reference frames from the motion library.
  - Clip ends are handled as in D8.
- **Objective:** Eq. 2 with the survival weight, plus the critic trained on λ-returns (Eq. 3: $\lambda = 0.95$, $\gamma = 0.99$, 5 epochs of 4 minibatches).
- **Optimizers:** the paper's, Adam with $\beta = (0.9, 0.95)$ for the policy.

All of this is needed by M4 anyway. M3 adds two checks, both with the PPO-Sim actor (same encoder–decoder network, D17) on the M2 model, because it is a good policy that stays on the model's data:

1. **Policy evaluation, no training.** Run the actor closed loop in imagination and in the simulator from the same held-out starts (same ξ, same observation history, no noise), and compare the returns over 32 steps.
   - M2 replayed logged actions. Here the actor reacts to observations computed from predicted states, so this checks the observation head, history, goals and termination together.
   - Pass: a return error close to M2's 1.1%.
2. **A short update run.** A few hundred BPTT iterations with the critic, with the policy evaluated in the simulator every 100.
   - It checks the update machinery: gradients reach every part of the policy, clipping and the critic fit behave, and the simulated return holds or improves while the policy is still near the data.
   - It is a test of our code, not a result about the method.

M3 gate: both checks pass. A bug is far easier to find here than inside the online loop.

M4: the online loop, as in the paper, from scratch on `dev`.

- **Each iteration:** about 2.5 simulator steps, 10 model batches (D10), one policy step and one critic fit.
- **Comparison:** PPO-Sim, on wall-clock time to a target return.
- **Ablations,** in order of value:
  1. without the critic: the paper's ablation, and the proof that our critic helps;
  2. $H = 16$, only if the imagined-vs-simulated gap grows.

A Settings and compute

World model	
state / action / context dims	77 / 28 / 32
dynamics MLP	3×2048, ELU; 80 outputs (state change + heading-frame pose change)
termination MLP	2×256, ELU; inputs: state + reference root height and g_z
normalization	per dimension, from 200k training transitions, frozen
pose loss scales	0.1 m, 0.1 rad
Training	
windows	$k = 8$, never across an episode end
optimizer	Adam, lr $2 \cdot 10^{-4}$, batch 2048 windows, gradient-norm clip 1.0, TF32
steps	50k; keep the best held-out 8-step loss (main: step 42k)
Data	
policies	PPO-Sim (1 Hz arms/waist) at iterations 1000, 4000, 8800
per policy	4096 envs $\times$ 500 steps, action noise $\mathcal{N}(0, 0.1^2)$, paper DR per episode
library / held-out	`dev` (1296 clips, 2.56 h) / 118 clips by name hash
GPU time (RTX 5060 Ti 16 GB, shared)	
collection	2 min
training	main 43, k=1 6, width 1024 16, absolute 43, unseen policy 43 min
evaluation	1 min open loop (all models), 33 s gradient check

B Reproduction

From the worktree root; GPU jobs go through the queue (`gpu-run`). Write the data paths out in full: the queue's shell does not split a variable into several arguments.

```
D="logs/supermimic_m2/data"; P="logs/rsl_rl/supermimic_ppo_sim_1hz/2026-10-22_21-11-56"
# 1. data (2 min)
python -m pal_mjlab.scripts.supermimic.collect_rollouts \
  --task Mjlab-SuperMimic-PPO-Flat-Pal-Kangaroo-Grippers-ArmWaist1Hz \
  --checkpoints $P/model_1000.pt $P/model_4000.pt $P/model_8800.pt --out-dir $D
# 2. models (main; ablations add --k 1 | --hidden 1024 1024 1024 | --no-predict-delta
#    | --train-sources model_1000 model_8800)
python -m pal_mjlab.scripts.supermimic.train_world_model \
  --data $D/model_1000 $D/model_4000 $D/model_8800 --out-dir logs/supermimic_m2/wm/main
# 3. open-loop evaluation and failure AUC (add --sources model_4000 for the policy shift)
python -m pal_mjlab.scripts.supermimic.eval_world_model \
  --models main=logs/supermimic_m2/wm/main/wm_best.pt k1=... \
  --data $D/model_1000 $D/model_4000 $D/model_8800 --out-json logs/supermimic_m2/eval/all.json
# 4. gradient check
python -m pal_mjlab.scripts.supermimic.grad_check --model logs/supermimic_m2/wm/main/wm_best.pt \
  --data $D/model_1000 $D/model_4000 $D/model_8800 --out-json logs/supermimic_m2/eval/grad_main.json
# 5. figures, tables, this report
python docs/supermimic/m2/make_figures.py && (cd docs/supermimic/m2 && latexmk -pdf report.tex)
```

Code.

- Model and data: `state.py`, `rl/world_model.py` and `rl/replay.py` in `src/pal_mjlab/tasks/supermimic/`.
- Scripts: `collect_rollouts.py`, `train_world_model.py`, `eval_world_model.py` and `grad_check.py` in `src/pal_mjlab/scripts/supermimic/`.
- Tests: `tests/supermimic/test_world_model.py`. They cover the state round trip, windows inside episodes, overfitting a batch, gradients against finite differences, and reward parity on collected data.